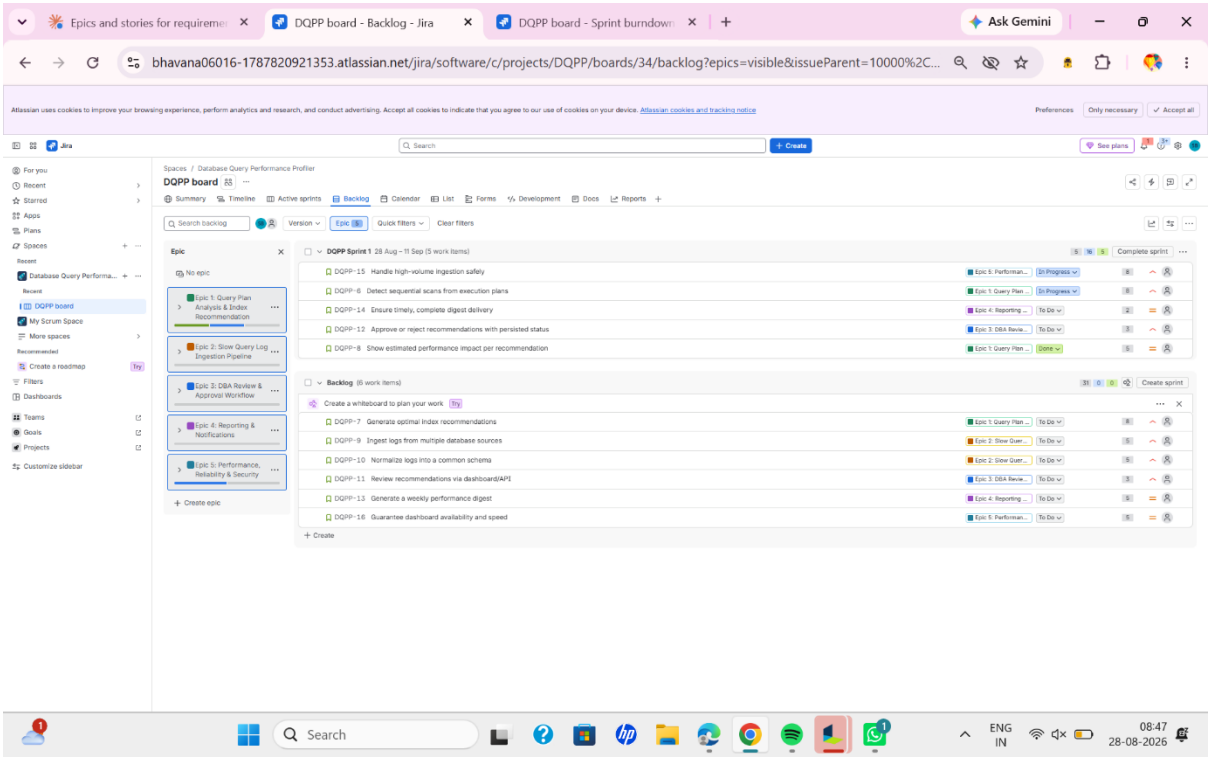
NAME : S V BHAVANA

SRN : PES2UG24CS417

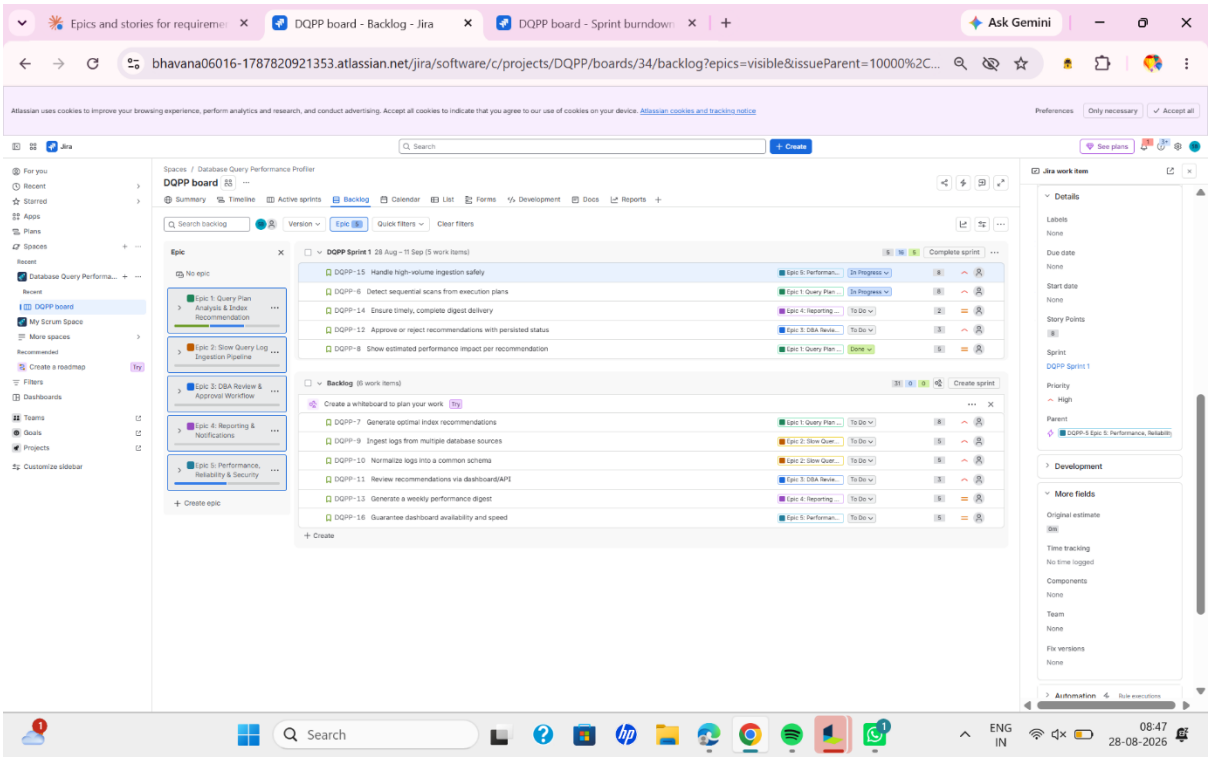
SECTION : G

SCREENSHOTS:

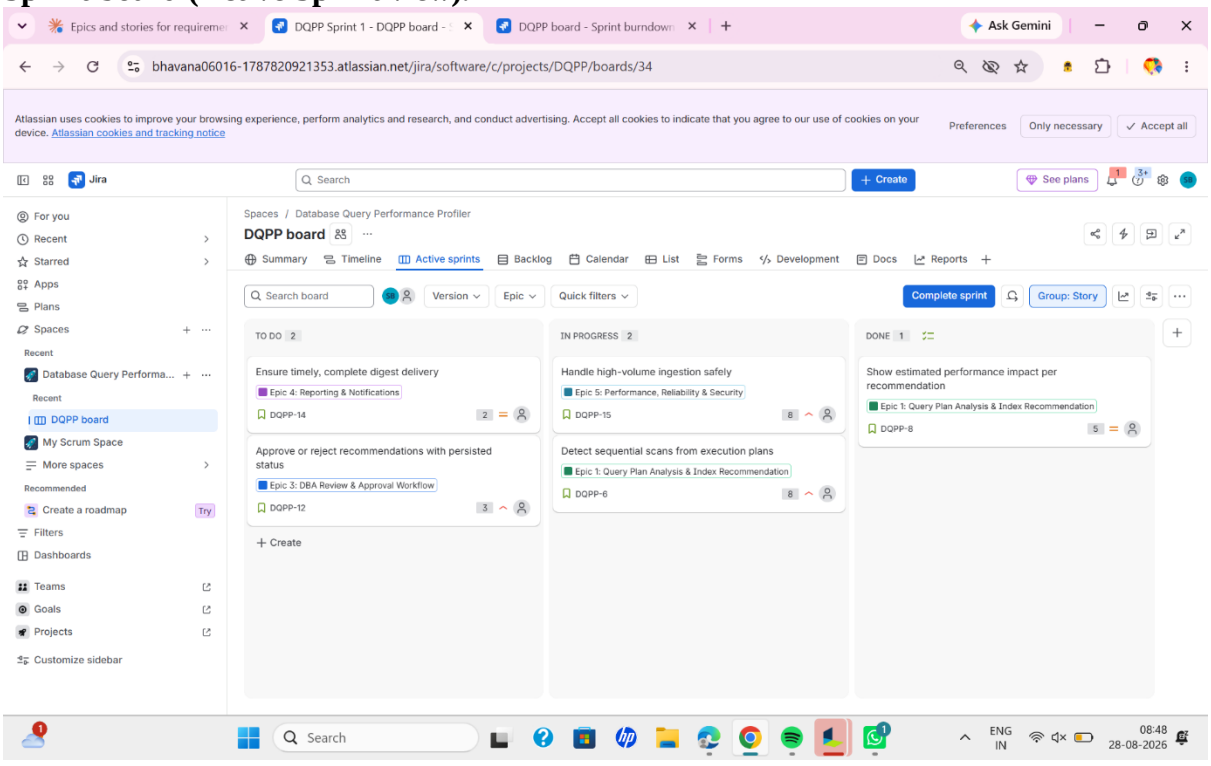
1. **Jira backlog with Epics and User Stories.**



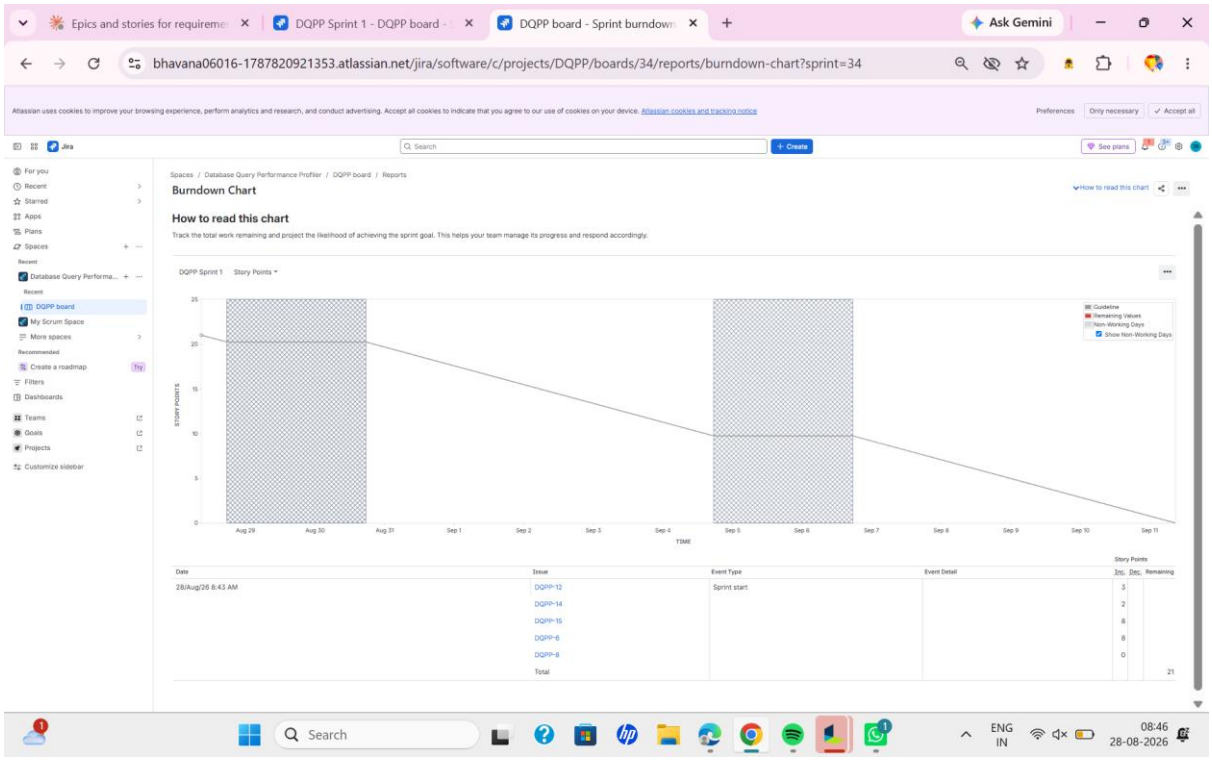
2. **Story point assignments.**



3. Sprint board (Active Sprint view).



4. Burndown Chart.



Reflection questions:

1. **Did your estimations reflect the actual effort?**
   Partially. Stories with well-understood mechanics — US-3.1, US-3.2 (dashboard status flip and persistence) — tracked close to their estimates, since they're standard CRUD-style work. But **US-1.1** (parsing EXPLAIN plans across both Postgres and MySQL) ran heavier than its 8-point estimate suggested: the two engines format plans differently enough that "one parser" became closer to two parsers plus a normalization layer. Similarly, **US-5.1** (5,000 records/min ingestion without host DB impact) is the kind of NFR that looks like a clean number on paper but usually surfaces edge cases only during load testing — actual effort tends to exceed estimate here too. Net effect: technical/algorithmic stories were mildly underestimated, while workflow/CRUD stories were estimated accurately.

2. **Was your backlog well-prioritized?**
   Mostly yes, in that priority followed the real dependency chain: ingestion (Epic 2) and plan parsing (Epic 1) had to come before recommendations could exist, and both were correctly marked High. One weak spot: **US-4.2** (digest delivery reliability) got slotted into Sprint 1 purely to fill leftover capacity rather than because it was logically ready — its parent story US-4.1 (digest generation) wasn't built until Sprint 3. That's a sequencing gap: a dependent story landed before its dependency. A tighter backlog would've kept Epic 4's two stories in the same sprint.

3. **How did your simulated sprint align with your plan?**
   Sprint 1 finished on time by day 10, but the shape of the burndown shows it wasn't a smooth landing — the team fell behind the ideal line through days 1–6 (still 11 points

remaining at the sprint's midpoint vs. an ideal of 8), then compressed the remaining work into the back half. So the *plan* (20 points, 10 days) held at the boundary, but the *path* to get there involved a late-sprint crunch rather than steady progress.

4. **What does the burndown chart tell us about team capacity?**
   The actual line consistently trails the ideal line for the first 60–70% of the sprint, then steepens sharply at the end. That pattern usually means the early days were absorbed by investigation/design work (e.g., understanding two different EXPLAIN plan formats for US-1.1) rather than points actually closing out — and the "catch-up" at the end is a sign of last-minute push rather than sustainable pace. Two implications for planning: (a) the team's *effective* velocity is probably a bit below the assumed 20 points — something like 17–18 is safer for Sprint 2/3 planning, and (b) front-loading a sprint with one large, unfamiliar technical story (like US-1.1) creates a slow start that the whole sprint has to compensate for later — splitting such stories into smaller ones (e.g., separate Postgres-parsing and MySQL-parsing tasks) would likely produce a straighter, more predictable burndown.